

High performance prototype design using lower bit RSA algorithm

Devakinandan Palla
School of Electronics Engineering
Vellore Institute of Technology
Vellore, India

Sourodipto Bhowmick
School of Electronics Engineering
Vellore Institute of Technology
Vellore, India

Abhishek Kumar
School of Electronics Engineering
Vellore Institute of Technology
Vellore, India

devakinandan.palla2019@vitstudent.ac.in sourodipto.bhowmick2019@vitstudent.ac.in abhishek.kumar2019@vitstudent.ac.in

Dr.Sakthivel Ramachandran
School of Electronics Engineering
Vellore Institute of Technology
Vellore, India
rsakthivel@vit.ac.in

Abstract—Today's modern technology in is all about having a tradeoff between various factors like power, area, performance, and as a result the objective ultimately is to cater to the needs of the desired customer. The project aims to have an outlook on compromising security for the sake of high performance. Thus, this can be used in applications which do not really have much security concerns or the risks of less performance, outweigh that of security issues. High frequency trading is a major area where the above requirements can be fulfilled. In high-frequency trading (HFT), financial institutions use powerful algorithms to make trades in fractions of a second. In order to achieve the necessary speed, security protocols may be relaxed, such as reducing encryption strength or using weaker authentication mechanisms. This can make HFT vulnerable to cyber-attacks, but the potential profits outweigh the risks for some traders, where this reduced key size can come into play. In order to achieve this goal, we have used a 64-bit RSA algorithm using the best suited multiplier algorithm, the Booth's multiplier and simple Square and Multiply algorithm. We have obtained an excellent throughput with a very good frequency along with a low-cost implementation due to its reduced complexity. We have also provided adequate methods to counter security concerns due to its extremely small key size, and therefore wish that our proposals shall be useful. The tools that we have used for the implementation are ModelSim, Intel Quartus II Prime(FPGA) using Verilog HDL code. We have also done a supplementary study on how Quantum computing has been successful in breaking the most complex of algorithms up to 1024 bits in hours, which has made us stick to a lower 64-bit encryption size with alternate proposals to enhance security while keeping the speed and performance intact. For this, we have used Qiskit software to study and observe.

Index Terms—RSA,Quartus Prime, Verilog HDL,Qiskit

I. INTRODUCTION

High performance precedence over security has many applications and fields demanding. To list a few, take High-Frequency Trading where high-frequency trading (HFT), financial institutions use powerful algorithms to make trades in fractions of a second. In order to achieve the necessary speed, security protocols may be relaxed, such as reducing encryption strength or using weaker authentication mechanisms. This can make HFT vulnerable to cyber-attacks, but the potential profits outweigh the risks for some traders

Another such application would be Video Streaming: Video streaming services require a lot of bandwidth to deliver high-quality video to users. To ensure smooth playback, video streaming services may sacrifice some security measures such as using weaker encryption algorithms or not using encryption at all. This can make the video stream vulnerable to eavesdropping, but it may be deemed an acceptable trade-off for delivering a seamless user experience.

We can also use it in the sensational Internet of Things (IoT) devices, such as smart home appliances and wearable technology. They often have limited processing power and memory. In order to achieve high performance, security measures such as encryption and authentication may be reduced or eliminated. This can make IoT devices vulnerable to cyber-attacks, but it may be seen as a necessary trade-off for providing a seamless user experience.

Another risky yet fair application would be mobile apps, which often require high performance to deliver a smooth user experience. To achieve this, some security measures may be compromised, such as storing sensitive data in plain text or using weak encryption algorithms. This can make mobile apps vulnerable to cyber-attacks, but it may be deemed acceptable if the user experience is improved.

It is important to note that in all of these cases, the potential risks should be carefully considered before compromising security measures for high performance. This is why there are many alternative techniques suggested to optimize the lost security action. Here are a few of them, and they can be used in accordance to the context and application required.

Usage of secure key exchange protocols: One way to mitigate the risks of a smaller RSA key size is to use a secure key exchange protocol, such as Diffie-Hellman key exchange or Elliptic Curve Diffie-Hellman key exchange, to establish a shared secret key between the communicating parties. This shared secret key can then be used to encrypt and decrypt messages, providing an additional layer of security.

Implementation of multi-factor authentication: multi-factor authentication can be used to ensure that only authorized users

are able to access the system or data being encrypted. This can include using passwords, biometrics, or other authentication factors in addition to the RSA key.

Usage of message authentication codes (MACs): A message authentication code (MAC) can be used to verify the authenticity of the message and ensure that it has not been tampered with. This can be particularly useful in situations where the key size is too small to provide strong encryption, as it provides an additional layer of security.

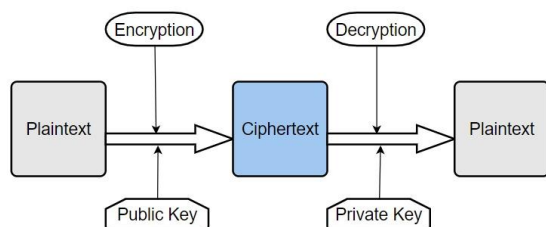
Usage of a trusted hardware module: A trusted hardware module, such as a hardware security module (HSM), can be used to store and manage RSA keys securely. This can help to protect the key from unauthorized access or tampering, even if the key size is small.

Implementation of access controls and monitoring: Access controls and monitoring can be used to ensure that only authorized users have access to the system or data being encrypted, and to detect and respond to any unauthorized access attempts or security breaches.

It's important to note that while these techniques can help to mitigate the risks associated with a smaller RSA key size, they are not a replacement for using a larger key size when strong security is required.

The reason we choose RSA is despite what we use is a lower bit application, its tougher ability to crack, allows us to challenge ourselves to break it down. For example, take the classic example of Alice and Bob. Alice wants to send Bob a message, and thus she encrypts the message with a security key. But unfortunately, Bob doesn't have access to the secret key to unlock it. Alice now is unable to send Bob the secret key securely, since it gets hackers an easy job to decrypt or corrupt the message. Thus, now what she does is, by using RSA algorithm, she makes both of them generate a pair of public key and private key, that are mathematically linked to each other on their computers. The public key is used to encrypt the data, whereas only the private key is used to decrypt the data. Even though the links are related to each other, one cannot be derived from each other. Therefore, what Alice and Bob now do is, exchange each other's public key, and then send the message after encryption. Thus, Bob is now able to open his message through his unique private key, and not even Alice can decrypt it.

We, thus used this above technique because despite our potential threats of using a small bit key size, the kind of algorithm which we used is branded and reputed enough and is still pretty strong enough.



II. LITERATURE REVIEW

Design and Analysis of Digital Circuits using Quantum Cellular Automata and Verilog- QCA-based digital

circuits can be designed using Verilog, which provides a standard

way of describing digital circuits. The combination of QCA and Verilog can potentially enable the development of novel digital circuits with improved performance characteristics, such as reduced power consumption and increased speed. However, there are also some shortcomings of using QCA and Verilog for digital circuit design and analysis: QCA circuits are highly sensitive to environmental factors such as temperature, which may affect their performance and reliability. Verilog does not directly support QCA-specific features such as cell polarization.^[1]

A Hardware-Accelerated Qubit Control System for Quantum Information Processing - Hardware acceleration can significantly improve the speed and precision of qubit control operations, enabling the implementation of more complex quantum algorithms. The proposed system can be integrated with existing quantum hardware platforms, such as superconducting qubits or trapped ions, to provide improved control capabilities. However, the system may be susceptible to errors due to noise and other environmental factors, which can have a significant impact on the performance of quantum algorithms. The integration of the control system with existing quantum hardware platforms may require significant engineering efforts and may pose compatibility issues.^[2]

Quantum Computer Architecture Toward Full-Stack Quantum Accelerators- A full-stack quantum accelerator can provide a comprehensive and integrated solution for quantum computing, incorporating hardware, software, and system-level components. The architecture can be designed to support a wide range of quantum algorithms and applications, including optimization, simulation, and cryptography. However, the hardware and software components of a full-stack quantum accelerator may require specialized expertise and resources, which may limit the accessibility and adoption of the technology. The development and optimization of quantum algorithms may require significant computational resources, which may be challenging to provide in the current technological landscape.^[3]

An FPGA-based real quantum computer emulator- Emulating quantum algorithm on a standard computer often requires much more time than it would on a quantum machine. Thus, we can shift the weight from time to hardware complexity. This is where we can take chances at field-programmable gate arrays, or FPGAs, come to help us to realize our target. However, it has following limitations: Limited qubit support: The FPGA-based emulator may be limited in the number of qubits it can simulate, which could limit the scope of experiments that can be performed with the emulator. Limited simulation accuracy: The emulator may not be able to simulate certain types of quantum circuits with high accuracy, which could impact the quality of experimental results obtained.^[4]

Quantum Computer Architecture for Quantum Accelerator- There are two important challenges in a quantum computing world. The first one is having enough better-quality qubits in the experimental quantum processor. The qubit technologies which are in good demand include ion traps, semiconducting and superconducting qubits, nitrogen vacancy-centres, etc. Making improvements on qubits is challenging as they suffer from de coherence that produces errors while performing quantum gate operation.

A High-Performance VLSI Architecture for Advanced Encryption

Standard (AES) Algorithm- This paper has finally presented a FPGA programming or hardware design.^[9]

long-searched VLSI implementation of this algorithm, with the desired features of high performance, high throughput and also high efficiency. The generation of subkeys generated by every round of the Rijndael algorithm(both encryption and decryption) is achieved by the expansion of the initial key, using the key scheduler module, along with pipelining. This implementation can be used for key lengths of 192 bits and 256 bits. While the AES algorithm is considered secure, hardware implementations may be vulnerable to side-channel attacks, such as timing or power analysis attacks, which could compromise the security of the encrypted data.^[5]

Cryptographic Attack Possibilities over RSA Algorithm through Classical and Quantum Computation-This paper has done a comparative analysis on 3 types of methods to factorize a given integer: Shor's Algorithm, Quadratic Sieve and Trial Division. While the Sieve's method does have a better efficiency to factor bigger integer compared Trial Division, Shor's Algorithm is the ultimate winner. This is because it possesses a challenge to the security parameters of the existent cryptosystem. The analysis in the paper is based on certain assumptions, such as the difficulty of factoring large numbers, which may not hold true in the future. Any new breakthrough in factoring large numbers could render the RSA algorithm vulnerable to attack.^[6]

Quantum Security Analysis of AES- To compute how complex the structure is, a framework is being proposed using both the classical and quantum method. After an experiment on various classical key recovery attacks, it is understood that the security margin of the AES 256 is bigger in the quantum world. However, there are few shortcomings. The analysis may be limited in the number of qubits used to simulate the AES algorithm, which could limit the scope of experiments that can be performed with the emulator. The analysis may not be able to simulate certain types of quantum attacks with high accuracy, which could impact the quality of the results obtained.^[7]

FPGA Based Hardware Abstraction of Quantum Computing System- The paper is a detailed proposal of resource efficient FPGA based quantum circuit abstraction. Thus, it is aimed to provide the FPGA based platform that constitutes fundamental sub blocks to design quantum circuits, using which it is aimed to implement an embedded system that can store, measuring, and introducing a phase shift in the q bit. The uniqueness of the proposed design is its flexibility to enhance the quantum circuit emulation accuracy. This however, comes at the cost of computational resources.^[8]

QubiC: An Open-Source FPGA-Based Control and Measurement System for Superconducting Quantum Information Processors - An open-source FPGA-based QubiC system bridging the quantum algorithm and the quantum hardware implementation layers of the computing stack is developed. Thus, it is demonstrated that QubiC is capable of running advanced quantum algorithms such as RC with a fairly decent execution speed. It can accommodate the full spectrum of users and developers and as a result, pose as a potential open-source toolbox for the quantum community. However, there are some major shortcomings: Complexity: The system may be complex and difficult to implement for researchers without experience in

FPGA-based electronic system for the control and readout of superconducting quantum processors- In this paper, an electronic system with a distributed synchronous clock architecture based on field-programmable gate array (FPGA). It is aimed to enable precise timing control, arbitrary waveform generation, IQ demodulation for qubit state discrimination, and the generation of real-time qubit-state-dependent trigger signals for feedback/feedforward control. However, The system may not have built-in support for quantum error correction, which could limit its usefulness in practical quantum computing applications.^[10]

New Hardware Architectures for Montgomery Modular Multiplication Algorithm- 2 hardware architectures are generally used for Montgomery multiplication. These architectures are based on new ideas where parallelism is enhanced by processing partial results using two different assumptions considering the most significant bit of each partial result word. Additionally, one of new Architecture consists of a new original data dependency graph which aims at significantly simplifying the control unit of each processing element. The proposed architectures may not offer significant improvements in performance over existing architectures, especially for small to medium-sized operands.^[11]

A New RSA Encryption Architecture based on Optimized Montgomery Multiplication- The proposed architecture can improve the efficiency and speed of RSA encryption by optimizing the multiplication operation using Montgomery multiplication. Montgomery multiplication can provide significant performance improvements over traditional multiplication algorithms, particularly for large integers used in RSA encryption. The optimization of the multiplication operation may require specialized expertise and resources, which may limit the accessibility and adoption of the technology. The proposed architecture may not be compatible with existing RSA encryption systems.^[12]

Fault and Simple Power Attack Simple RSA using Montgomery Modular Multiplication- FA and SPA resistance makes a valuable component in an RSA hardware module. However, this feature is considered expensive when computational cost comes into play. An algorithm for FA and SPA resistant modular exponentiation has been designed for RSA that is based on an advanced version of Carry Save Montgomery modular multiplication algorithm. Using the given algorithms, a Montgomery modular multiplication architecture is used to design a FA and SPA resistant modular exponentiation architecture. However, this paper does not provide a detailed evaluation of the effectiveness of the proposed attack in practice, including the success rate or complexity of the attack.^[13]

High-Speed and Scalable FPGA Implementation of the Key Generation for the Leighton-Micali Signature Protocol- To improve the parallelism and scalability and decrease the latency, several hardware architectural optimizations are done. The flow of control is well designed to fit any different parameter sets with constant power. The experimental results are carried out to prove that the new hardware implementation can easily replace the conventional optimal software implementation. However, the paper does not provide a comparison of the proposed architecture with other state-of-the-art implementations of the key generation

process for the Leighton-Micali signature protocol or other signature protocols.^[15]

III. BASICS OF RSA ALGORITHM

RSA (Rivest–Shamir–Adleman) is a public-key encryption algorithm widely used for secure data transmission. It is named after its inventors Ron Rivest, Adi Shamir, and Leonard Adleman.

In the RSA algorithm, each user has a pair of keys: a public key and a private key. The public key is shared with everyone, while the private key is kept secret. The public key is used to encrypt messages, while the private key is used to decrypt messages.

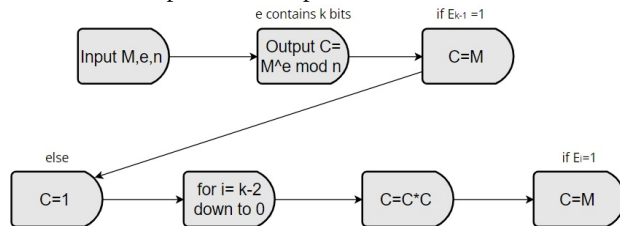
The algorithm works as follows:

Choose two large prime numbers p and q .
 Calculate $n = p \cdot q$.
 Calculate $\phi(n) = (p-1) \cdot (q-1)$.
 Choose a public exponent e such that $1 < e < \phi(n)$ and e is coprime with $\phi(n)$.
 Calculate a private exponent d such that $d \cdot e \equiv 1 \pmod{\phi(n)}$.
 The public key is (n, e) .
 The private key is (n, d) .
 To encrypt a message m , compute $c = m^e \pmod{n}$.
 To decrypt a ciphertext c , compute $m = c^d \pmod{n}$.

IV. OVERVIEW OF RSA ALGORITHM

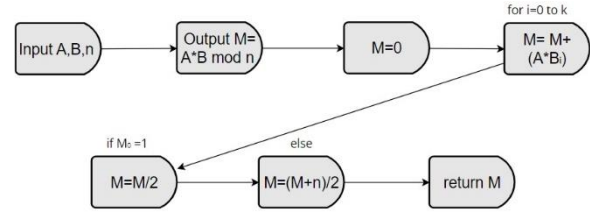
RSA encryption and decryption are mutual inverses and commutative due to symmetry which is present in modular arithmetic. Hence the Encryption engine contains both the operation of Encryption and Decryption. Here is the mathematics involved in modular arithmetic: A and B are congruent modulo m if and only if $A - B$ is divisible by m . It is written as: $A \equiv B \pmod{m}$ Where m is a positive integer known as modulus. The sign " $\equiv$ " indicates congruence.

Modular exponentiation operation has a series of modular multiplication and squaring operation. This simplification is based on an algorithm called **square and multiply algorithm**. This algorithm is based on scanning the bit of the exponent from the left to the right. In every iteration, i.e., for every exponent bit, the current result is squared when the currently scanned exponent bit has the value 1, a multiplication of the current result by M is obtained following the squaring. This algorithm can be represented in pseudo code as shown below.



The modular multiplication problem is the computation of $P = A \times B \pmod{n}$, with integers A , B , and n . It is usually considered that A and B are positive integers with $0 \leq A, B < n$. The square or multiplication operation is just a simple

multiplication process. There are many good approaches to perform multiplication such as Multiply then divide, Interleaving multiplication and reduction, etc. But here Montgomery's algorithm is used. It avoids the "division" operation and uses "shift and addition" operations to do modular multiplication. Suppose A and B are two k -bit positive integers, respectively. Let A_i and B_i are the bit of A and B , respectively. The algorithm is as follows:



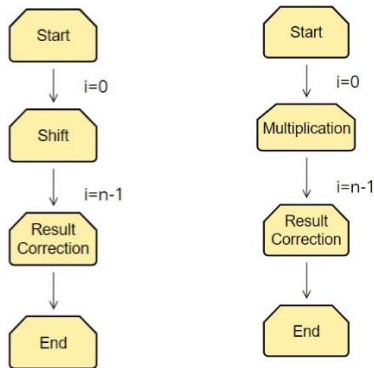
In this paper carry save adder is used for performance of both addition and subtraction operation. Subtraction means addition of a number with 2's complement of one more number. Here a select lines are used to perform the selection of adder or subtracter. If A , B are positive integers, and S is the result then $S = A + B$, when select line is 0 else $S = A - B$ will be the output

V. PROPOSED METHODOLOGY

For obtaining the output and waveforms on Verilog, we have implemented the code in 4 different submodules with modular operations being the main one. The modular operations code defines a module named "mod_operationrsa" that implements a basic arithmetic operation using two 32-bit input values "in1" and "in2". The module has inputs for clock and reset signals, as well as other control signals such as "op_rdy" and "sign". The output of the module is a 32-bit value named "out" and an output signal named "out_rdy", which indicates when the output is ready. The module uses a set of registers and wires to implement the arithmetic operation, including a register for the output value, a register for a temporary output value, and registers for the input values. Within this block, the module uses a set of if-else statements to perform the arithmetic operation based on the input values and control signals. Overall, this Verilog HDL code implements a basic arithmetic operation that can be used in RSA encryption and decryption algorithms. It covers both encryption and decryption engines

Another module seems to implement the RSA algorithm for encryption. Here is a brief overview of the input and output signals and their roles in the module:

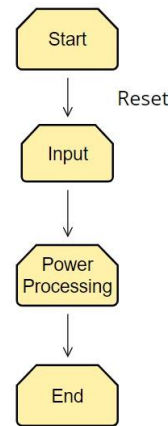
Input Signals	Output Signals
ENCRY_DATA	the encrypted message
ENCRY_RDY	a signal indicating that the ENCRY_DATA output is valid
mod_num1,2	registers holding the input to the modular operation
power_num1,2	registers holding the input to the power operation
mod_rdy	a signal indicating that the modular operation has completed
mod_out	the output of the modular operation
power_out:	the output of the power operation
power_rdy:	a signal indicating that the power operation has completed
count	a counter used to keep track of the number of iterations in the power operation
power_value	a register holding the output of the power operation
mod_value	register holding the output of the modular operation



Another module seems to implement a power function that computes data1 raised to the power of data2. The inputs to the module include a clock signal (clk), a reset signal (reset), two data inputs (data1 and data2), and an input ready signal (in_rdy). The outputs of the module include the output signal (out) and an output ready signal (out_rdy). The module uses a sequential always block triggered on the positive edge of the clock signal. The loop iterates data2 times, multiplying temp by data1 on each iteration. The count variable keeps track of the number of iterations performed so far. If count reaches

clk	the clock signal
reset	the reset signal
KU1	the first part of the public key
KU2	the second part of the public key
MESSAGE	the message to be encrypted
data_rdy	signal indicating that the MESSAGE input is ready for processing

data2, the loop terminates and out_rdy is set to true. Otherwise, out_rdy is kept false. Once the loop is complete, the final result is stored in temp, which is then output on the out signal



For obtaining the quantum circuit on Qiskit, Quantum Phase Estimation Algorithm plays a vital role. The code starts by defining some parameters required for RSA encryption such as the prime numbers p and q, the public exponent e, the modulus n, and the totient of n (phi n). It then defines a message to be encrypted. The code then creates a quantum circuit with a number of qubits equal to the bit length of n (num bits), and one less classical bit (num bits_1) to store the estimated phase. The message is encrypted by computing the ciphertext c using the RSA encryption formula ($c = \text{message}^e \bmod n$). The binary representation of the ciphertext is then obtained and the qubits corresponding to the binary digits of 1 are put into a superposition state using Hadamard gates. The next step is to perform the quantum phase estimation algorithm (QPE) to estimate the phase angle corresponding to the eigenvector of a unitary operator. In this case, the unitary operator is chosen to be a controlled rotation operator that rotates the phase of the state of the last qubit by an angle proportional to the binary representation of the ciphertext. The angle of rotation is chosen to be $\theta = 2\pi / (2^{\text{num_bits}})$.

VI. SIMULATION RESULTS AND DISCUSSION

Following are results for the RSA algorithm implemented on Verilog.

Numerical Analysis of the ModelSim Testbench:

We have taken 2 prime numbers P= 5 and Q= 7, along with a message signal M=2:

Now calculating for N= P*Q= 5*7=35

$$\Phi(N)=(P-1)(Q-1)= 4*6= 24$$

Next, we choose e such that $\text{GCD}\{\Phi(N),e\}= 1$; $24=2*2*2*3$;
e cannot be 2,3,4, choose e=5; [Greatest Common Divisor]

Next, we find the encrypted Cipher text by applying

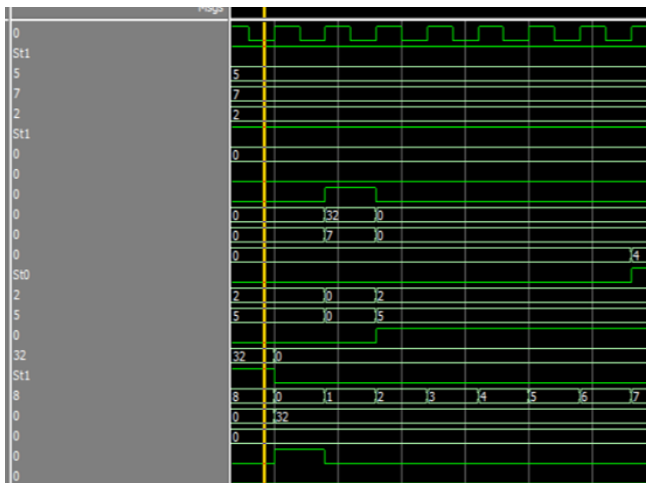
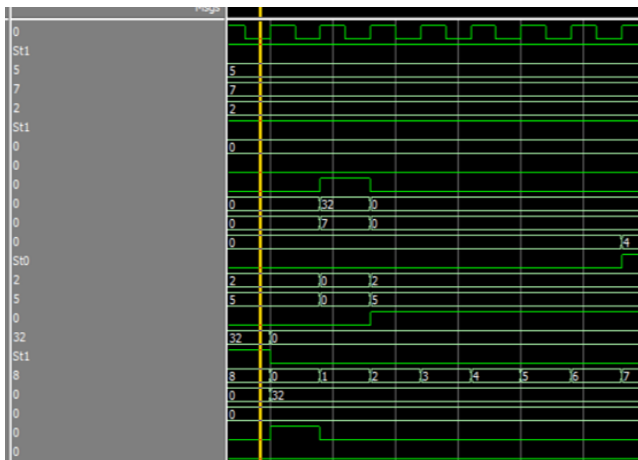
$$C= (M^e)\text{mod}(N):$$

$$(2^5)\text{mod}35= 32\text{mod}35 = 32;$$

Thus, the encrypted value we obtained is 32(or)100000.

Like the previous case,

We gather prime numbers, P=3, Q= 23 and a message signal
M=6



Corresponding Dataflow:

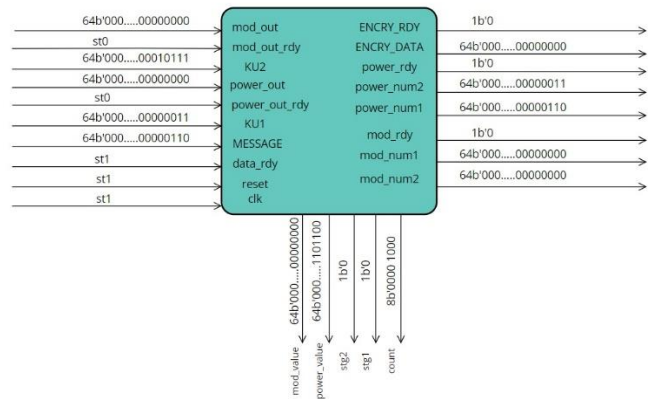


Table 1. Analysis and Synthesis Usage Report Summary

Component	Numbers
Estimate of Logic utilization (ALMs needed)	216
Registers	195
Total I/O Pins	197
Input ports	68
Output ports	33
Combinational ALUT usage for logic	301
Total fan-out	2389
Number detected on machine	1231
Maximum allowed	8
Processor 1, 2-4	100%,0%
ALMs used for LUT logic	30

Table 2. Timings Analyzer(Clocks)

Period	1000
Maximum Frequency	1000 MH
Rise	0.00
Fall	0.500
Speed Grade	4

Table 3. Register Statistics

Number of registers using Synchronous Clear	195
Number of registers using Synchronous Load	193
Number of registers using Clock Enable	192
Number of registers using Asynchronous Clear Number of registers using Asynchronous Load	0

From the comparative analysis of above papers, we have inferred the following:

The first paper involves 128 bit and used Bit-Serial Systolic Algorithm. Our inference from this comparison was our sample had a higher frequency, lesser slices, LUTs and lower time period.

The second paper involves 64 bit and used right to left algorithm. Our inference from this comparison was our sample had a higher frequency, lesser slices, LUTs and lower time period.

The third paper involves 128-bit simple shift and add algorithm. Our inference from this comparison was our sample had a higher frequency, lesser slices, LUTs and lower time period.

The fourth paper involves 1024-bit secured communication system based on Montgomery algorithm. Our inference from this comparison was our sample had a higher frequency, lesser slices, LUTs and lower time period.

The fifth paper involves 64-bit hardware base application RSA encryption algorithm. Our inference from this comparison was our sample had a higher frequency, lesser slices, LUTs and lower time period.

We have tabulated these comparisons and have also obtained percentage increase/decrease with respect to our findings:

Table 4. Comparison Parameters from our work

Parameter	Value
Maximum Frequency	1000 MHz
Number of slice flip flops	195
Number of LUTs	301
Period	1.000 ns
Speed Grade	4

Maximum Frequency: It refers to the maximum clock speed at which a digital circuit can operate without violating its timing requirements. It is typically measured in MHz or GHz.

Number of slice flip flops: Slices are the basic building blocks of digital circuits in FPGA (Field Programmable Gate Arrays). A slice flip-flop is a memory element that can store a single binary bit value, and it is used to synchronize the data with the clock. The number of slice flip-flops in an FPGA design is an important metric that affects its performance and resource utilization.

Number of LUTs: LUT (Lookup Table) is a small memory unit used to store the logic functions in digital circuits. The number of LUTs in an FPGA design is a measure of its complexity and resource utilization.

Period: It refers to the time taken by a clock signal to complete one cycle. The period is the inverse of the clock frequency and is typically measured in nanoseconds.

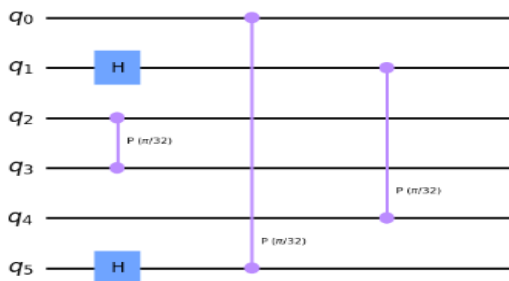
Speed Grade: In FPGA design, the speed grade refers to the maximum operating frequency at which an FPGA device can operate. The higher the speed grade, the faster the device can operate.

Parameters	Original	Paper-1[16]	Paper-2[17]	Paper-3[18]	Paper-4[19]	Paper-5[20]
Maximum Frequency	1000 MHz	101.061 MHz	58 MHz (128 bits)	81.06 MHz	69.093MHz	79.54 MHz
Number of Slice flip flops	195	2366	432	2943	8956	277
Number of LUTs	301	4325	568	4325	8032	3083
Period	1.000 ns	9.85 ns	12.473 ns	9.879ns	14.473ns	12.60 ns
Speed Grade	-4	-4	-4	-4	Nil	Nil

Comparative percentage increase of original paper	Paper-1[16] (%)	Paper-2[17] (%)	Paper-3[18] (%)	Paper-4[19] (%)	Paper-5[20] (%)
Maximum Frequency	888.939	1620.69	1137.48%	1343.83	1155.88
Number of Slice flip flops	-91.76	-54.86	-93.43	-97.83	-29.64
Number of LUTs	-93.045	-47.07	-93.03	-96.255	-90.263
Period	-89.847	-91.9827	-89.88	-93.0096	-92.06
Speed Grade	same	same	same	-	-

After the above section, we have done a brief study on quantum computing and its ability to break down high bit algorithms. We couldn't expand this section of our project due to the limitations of the Qubits allowed. The following results have been obtained from on Qiskit when we gave a simple test input to it.

```
Original message: 11
Encrypted message:16
Decrypted message: 11
```

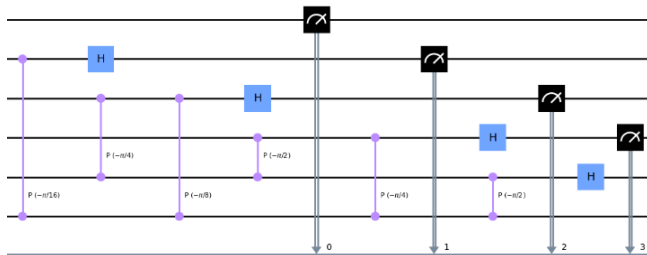


Here we have taken $p = 5$

$q = 7$

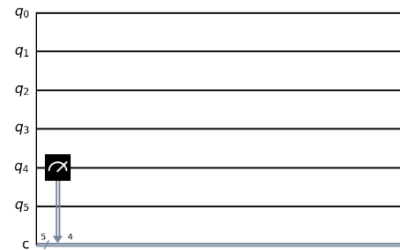
$e = 23$

The circuit generated by the circuit drawer function visualizes the quantum circuit that implements the RSA encryption scheme using the Quantum Phase Estimation (QPE) algorithm. The circuit has a total of num bits qubits, where the first num bits-1 qubits are used to perform the QPE and the last qubit is used to implement the RSA encryption. The circuit starts by applying the Hadamard gate to the qubits corresponding to the '1' bits in the binary representation of the ciphertext. Then, the Hadamard gate is applied to the qubit corresponding to the public key parameter 'e'.



Finally, the Inverse Quantum Fourier Transform (QFT) is applied to obtain the phase estimation result, which is used to calculate the decryption key using the multiplicative inverse function. The decrypted message is then obtained by applying the RSA decryption algorithm using the calculated decryption key.

Overall, the circuit implements the quantum part of the RSA encryption and decryption scheme, showing how quantum algorithms can be used to break classical encryption schemes.



VII.CONCLUSION

The Verilog level coding for RSA Encryption algorithm is done in 4 different modules. The desired waveforms have been obtained from the values in test bench. The maximum clock frequency is found to be about . We have implemented this in accordance to the fact that we are compromising security for the rewards of higher performance. As a result, we have used the best possible multiplier algorithm for the number of bits it is used. The key size although low can be compensated by techniques such as usage of secure key protocols, implementation of a multi factor authentication, usage of message authentication codes or a trusted hardware module. We have further proved our point by taking 5 samples of various bit sizes, and upon comparison, we have found out that our samples took less area and provided high frequency. Due to its less complexity, we have concluded that this design consumes less power, has a higher performance, but at the cost of good security. We have also studied how quantum computing has broke down every bit size up to 1024 bits, making every one of them vulnerable to be broken down in few hours or sometimes even minutes.

While this design sample has a fair share of advantages, it is still not recommended for its security concerns, and it is thus expected for the user to compensate the low bit key size with the above recommended compensation techniques, given in the introduction section for lost security aspect, while also keeping it appropriate by keeping the respective context in mind.

Regarding the quantum study, python code has been implemented in Qiskit to obtain the Encrypted and Decrypted message. A quantum level circuits are obtained using quantum phase estimation and Fourier Transforms.

VIII.ACKNOWLEDGEMENT

We would like to express our special thanks and appreciation to our project guide Dr.Sakthivel Ramachandran for his utmost patience in guiding us, supporting and encouraging us throughout this work and leading us in the right path. We acknowledge the valuable time he has put in for the fulfilment of our project.

IX. REFERENCES

- [1] V. M. Nanditha, U. Sanath Rao, M. Murali, R. Swathi and K. Chethana, "Design and Analysis of Digital Circuits using Quantum Cellular Automata and Verilog," *2020 7th International Conference on Computing for Sustainable Global Development (INDIACom)*, New Delhi, India, 2020, pp. 205-210, doi: 10.23919/INDIACom49435.2020.9083709.
- [2] N. Messaoudi, C. Crocker and M. Almendros, "A Hardware-Accelerated Qubit Control System for Quantum Information Processing," *2020 XXXV Conference on Design of Circuits and Integrated Systems (DCIS)*, Segovia, Spain, 2020, pp. 1-5, doi: 10.1109/DCIS51330.2020.9268643.
- [3] K. Bertels *et al.*, "Quantum Computer Architecture: Towards Full-Stack Quantum Accelerators," *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Grenoble, France, 2020, pp. 1-6, doi: 10.23919/DATE48585.2020.9116502.
- [4] Pilch, J., Długopolski, J. An FPGA-based real quantum computer emulator. *J Comput Electron* **18**, 329–342 (2019). <https://doi.org/10.1007/s10825-018-1287-5>
- [5] N. M. Kosaraju, M. Varanasi and S. P. Mohanty, "A high-performance VLSI architecture for advanced encryption standard (AES) algorithm," *19th International Conference on VLSI Design held jointly with 5th International Conference on Embedded Systems Design (VLSID'06)*, Hyderabad, India, 2006, pp. 4 pp.-, doi: 10.1109/VLSID.2006.9.
- [6] K. K. Soni and A. Rasool, "Cryptographic Attack Possibilities over RSA Algorithm through Classical and Quantum Computation," *2018 International Conference on Smart Systems and Inventive Technology (ICSSIT)*, Tirunelveli, India, 2018, pp. 11-15, doi: 10.1109/ICSSIT.2018.8748675.
- [7] Bonnetain, X., Naya-Plasencia, M., & Schrottenloher, A. (2019). Quantum Security Analysis of AES. *IACR Transactions on Symmetric Cryptology*, 2019(2), 55–93
- [8] Khalid, M., Mujahid, U., Jafri, A. *et al.* An FPGA-based hardware abstraction of quantum computing systems. *J Comput Electron* **20**, 2001–2018 (2021).
- [9] Y. Xu *et al.*, "QubiC: An Open-Source FPGA-Based Control and Measurement System for Superconducting Quantum Information Processors," in *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1-11, 2021, Art no. 6003811, doi: 10.1109/TQE.2021.3116540.
- [10] Yuchen Yang, Zhongtao Shen, Xing Zhu, Ziqi Wang, Gengyan Zhang, Jingwei Zhou, Xun Jiang, Chunqing Deng, Shubin Liu
- [11] M. Huang, K. Gaj and T. El-Ghazawi, "New Hardware Architectures for Montgomery Modular Multiplication Algorithm," in *IEEE Transactions on Computers*, vol. 60, no. 7, pp. 923-936, July 2011, doi: 10.1109/TC.2010.247.
- [12] A. P. Fournaris and O. Koufopavlou, "A new RSA encryption architecture and hardware implementation based on optimized Montgomery multiplication," *2005 IEEE International Symposium on Circuits and Systems*, Kobe, Japan, 2005, pp. 4645-4648 Vol. 5, doi: 10.1109/ISCAS.2005.1465668.
- [13] A. P. Fournaris, "Fault and simple power attack resistant RSA using Montgomery modular multiplication," *Proceedings of 2010 IEEE International Symposium on Circuits and Systems*, Paris, France, 2010, pp. 1875-1878, doi: 10.1109/ISCAS.2010.5537879.
- [14] Bertels, Koen & Ashraf, Imran & Nane, Razvan & Fu, Xiang & Rieseboos, L. & Varsamopoulos, Savvas & Mouedenne, A. & van Someren, Hans & Sarkar, Aritra & Khammassi, Nader. (2019). Quantum Computer Architecture: Towards Full-Stack Quantum Accelerators.
- [15] Y. Song, X. Hu, W. Wang, J. Tian and Z. Wang, "High-Speed and Scalable FPGA Implementation of the Key Generation for the Leighton-Micali Signature Protocol," *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, Daegu, Korea, 2021, pp. 1-5, doi: 10.1109/ISCAS51556.2021.9401177.
- [16] Sahu, Sushanta & Pradhan, Manoranjan. (2011). FPGA Implementation of RSA Encryption System. *International Journal of Computer Applications*. 19. 10-12. 10.5120/2391-3173.
- [17] Shams, Rehan & Khan, Fozia & Umair, Mohammad. (2013). Cryptosystem An Implementation of RSA Using Verilog. *International Journal of Computer Networks and Communications Security*. 1. 102-109.
- [18] Yuliang, Deng and Mao Zhigang. "FPGA Implementation of RSA Encryption System." (2011).
- [19] Ankit Anand, Pushkar Praveen, 2012, Implementation of RSA Algorithm on FPGA, *INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH & TECHNOLOGY (IJERT)* Volume 01, Issue 05 (July 2012)
- [20] Jyoti Kalia¹ and Vikas Mittal² ¹PG scholar, ¹Department of Electronics and Communication Engineering, MMU University, Mullana – 133207, Haryana, India. ²Department of Electronics and Communication Engineering, MMU University, Mullana – 133207, Haryana, India *International Journal of Electronics Engineering Research*. ISSN 0975-6450 Volume 9, Number 8 (2017) pp. 1329-1338

